

Manual de Implantação

INCIDENT_SYS — Sistema Web Seguro para Registro e Classificação de Incidentes de Segurança Cibernética

Versão: 2.0

Data: Abril de 2026

Repositório: https://github.com/margefson/incident_security_system

Modo de execução: Desenvolvimento local (localhost)

Equipe: Nattan, Keven, Margefson, Josias

Sumário

1. [Pré-requisitos](#)
2. [Clonagem do Repositório](#)
3. [Configuração do Banco de Dados MySQL](#)
4. [Configuração das Variáveis de Ambiente](#)
5. [Instalação das Dependências Node.js](#)
6. [Migração do Schema do Banco de Dados](#)
7. [Configuração do Ambiente Python \(Modelo ML\)](#)
8. [Treinamento do Modelo de Machine Learning](#)
9. [Inicialização dos Servidores Flask \(ML e PDF\)](#)
10. [Inicialização do Servidor Node.js](#)
11. [Verificação do Sistema](#)
12. [Scripts Disponíveis](#)
13. [Estrutura de Diretórios](#)
14. [Arquitetura de Serviços](#)

15. [Solução de Problemas Comuns](#)
16. [Divisão de Atividades do Grupo](#)

1. Pré-requisitos

Antes de iniciar a implantação, certifique-se de que os seguintes softwares estão instalados e funcionando corretamente na sua máquina.

1.1 Softwares Obrigatórios

Software	Versão Mínima	Versão Testada	Download
Node.js	18.x LTS	22.13.0	https://nodejs.org
pnpm	8.x	10.4.1	<code>npm install -g pnpm</code>
Python	3.9+	3.11.0	https://python.org
MySQL	8.0+	8.0	https://dev.mysql.com/downloads/
Git	2.x	qualquer	https://git-scm.com

1.2 Verificação das Versões

Execute os seguintes comandos no terminal para confirmar as versões instaladas:

```
node --version      # Deve exibir v18.x.x ou superior
pnpm --version      # Deve exibir 8.x.x ou superior
python3 --version   # Deve exibir Python 3.9.x ou superior
mysql --version     # Deve exibir mysql Ver 8.0.x ou superior
git --version       # Deve exibir git version 2.x.x
```

1.3 Alternativa ao MySQL: TiDB Serverless (Gratuito)

Caso não queira instalar o MySQL localmente, é possível utilizar o **TiDB Serverless** (compatível com MySQL) na camada gratuita em <https://tidbcloud.com>. Após criar o

cluster, copie a connection string fornecida no formato `mysql://usuario:senha@host:porta/banco?ssl=true`.

2. Clonagem do Repositório

Abra o terminal e execute os seguintes comandos para clonar o repositório e acessar o diretório do projeto:

```
git clone https://github.com/margefson/incident_security_system.git
cd incident_security_system
```

A estrutura de diretórios após a clonagem será detalhada na seção 13.

3. Configuração do Banco de Dados MySQL

3.1 Criação do Banco de Dados

Conecte-se ao MySQL com um usuário que possua privilégios de criação de banco de dados e execute os seguintes comandos SQL:

```
-- Criar o banco de dados
CREATE DATABASE incident_security CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;

-- Criar um usuário dedicado (recomendado para segurança)
CREATE USER 'incident_user'@'localhost' IDENTIFIED BY 'SuaSenhaForte123!';

-- Conceder privilégios ao usuário no banco criado
GRANT ALL PRIVILEGES ON incident_security.* TO 'incident_user'@'localhost';

-- Aplicar as alterações de privilégios
FLUSH PRIVILEGES;
```

3.2 Formato da Connection String

A connection string do MySQL deve seguir o formato abaixo. Ela será utilizada na variável de ambiente `DATABASE_URL` na próxima seção:

```
mysql://incident_user:SuaSenhaForte123!@localhost:3306/incident_security
```

Atenção: Substitua `SuaSenhaForte123!` pela senha definida no passo anterior. Se o MySQL estiver em uma porta diferente da padrão (3306), ajuste o número da porta na connection string.

4. Configuração das Variáveis de Ambiente

Na raiz do projeto, crie um arquivo `.env` com o seguinte conteúdo. Cada variável é explicada na tabela abaixo.

```
# Criar o arquivo .env na raiz do projeto  
touch .env
```

Abra o arquivo `.env` em um editor de texto e adicione as seguintes variáveis:

```
# — Banco de Dados —————
DATABASE_URL=mysql://incident_user:SuaSenhaForte123!@localhost:3306/incident_secu

# — Autenticação JWT —————
# Gere uma string aleatória segura de pelo menos 32 caracteres
JWT_SECRET=sua_chave_secreta_muito_longa_e_aleatoria_aqui_32chars

# — OAuth (Autenticação) —————
# Necessário apenas para login via OAuth externo. Para uso local com bcryptjs,
# pode ser deixado vazio ou com valores fictícios.
VITE_APP_ID=local-dev
OAUTH_SERVER_URL=https://oauth.seudominio.com
VITE_OAUTH_PORTAL_URL=https://auth.seudominio.com
OWNER_OPEN_ID=local-owner
OWNER_NAME=Admin Local

# — APIs Externas (Opcional) —————
# Necessário apenas para funcionalidades avançadas (notificações, LLM).
# Para uso local básico, pode ser deixado vazio.
BUILT_IN_FORGE_API_URL=
BUILT_IN_FORGE_API_KEY=
VITE_FRONTEND_FORGE_API_KEY=
VITE_FRONTEND_FORGE_API_URL=

# — Analytics (Opcional) —————
VITE_ANALYTICS_ENDPOINT=
VITE_ANALYTICS_WEBSITE_ID=

# — Aplicação —————
VITE_APP_TITLE=INCIDENT_SYS
VITE_APP_LOGO=
```

4.1 Descrição das Variáveis

Variável	Obrigatória	Descrição
DATABASE_URL	Sim	Connection string completa do MySQL
JWT_SECRET	Sim	Chave secreta para assinatura dos tokens JWT de sessão
VITE_APP_ID	Não	ID da aplicação no provedor OAuth
OAUTH_SERVER_URL	Não	URL do servidor OAuth externo
VITE_OAUTH_PORTAL_URL	Não	URL do portal de login OAuth
OWNER_OPEN_ID	Não	OpenID do proprietário (promovido a admin automaticamente)
OWNER_NAME	Não	Nome do proprietário
BUILT_IN_FORGE_API_*	Não	APIs externas opcionais (notificações, LLM)
VITE_ANALYTICS_*	Não	Analytics de uso (opcional)

4.2 Gerando o JWT_SECRET

Para gerar uma chave segura, execute um dos seguintes comandos:

```
# Usando Node.js
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"

# Usando OpenSSL
openssl rand -hex 32

# Usando Python
python3 -c "import secrets; print(secrets.token_hex(32))"
```

5. Instalação das Dependências Node.js

Com o arquivo `.env` configurado, instale todas as dependências do projeto:

```
# Na raiz do projeto
pnpm install
```

Este comando instala todas as dependências listadas no `package.json`, incluindo React, tRPC, Drizzle ORM, bcryptjs, Joi, Recharts e demais bibliotecas. O processo pode levar de 1 a 3 minutos dependendo da velocidade da conexão.

Nota: O projeto utiliza `pnpm` como gerenciador de pacotes. Não utilize `npm install` ou `yarn install`, pois o arquivo `pnpm-lock.yaml` garante versões exatas das dependências.

6. Migração do Schema do Banco de Dados

Com o banco de dados criado e o `DATABASE_URL` configurado, execute a migração para criar as tabelas necessárias:

```
pnpm db:push
```

Este comando executa internamente `drizzle-kit generate && drizzle-kit migrate`, que:

1. Gera os arquivos de migração SQL com base no schema definido em `drizzle/schema.ts`.
2. Aplica as migrações no banco de dados, criando as tabelas `users` e `incidents`.

6.1 Tabelas Criadas

Após a migração bem-sucedida, as seguintes tabelas estarão disponíveis no banco:

Tabela `users`

Coluna	Tipo	Descrição
id	INT AUTO_INCREMENT PK	Identificador único
openId	VARCHAR(64) UNIQUE	Identificador OAuth ou prefixo local_
name	TEXT	Nome completo do usuário
email	VARCHAR(320)	Endereço de e-mail
passwordHash	VARCHAR(255)	Hash bcrypt da senha
loginMethod	VARCHAR(64)	Método de login (local)
role	ENUM('user','admin')	Perfil de acesso
isActive	BOOLEAN	Status da conta
createdAt	TIMESTAMP	Data de criação
updatedAt	TIMESTAMP	Data da última atualização
lastSignedIn	TIMESTAMP	Data do último login

Tabela incidents

Coluna	Tipo	Descrição
id	INT AUTO_INCREMENT PK	Identificador único
userId	INT NOT NULL	FK para <code>users.id</code>
title	VARCHAR(255)	Título do incidente
description	TEXT	Descrição detalhada
category	ENUM	Categoria classificada pelo ML
riskLevel	ENUM('critical','high','medium','low')	Nível de risco
confidence	FLOAT	Score de confiança do modelo (0–1)
createdAt	TIMESTAMP	Data de registro
updatedAt	TIMESTAMP	Data da última atualização

7. Configuração do Ambiente Python (Modelo ML)

O motor de classificação automática requer Python 3.9+ com as bibliotecas `scikit-learn`, `Flask` e `openpyxl`.

7.1 Criação de Ambiente Virtual (Recomendado)

```
# Navegar para o diretório ml
cd ml

# Criar ambiente virtual
python3 -m venv venv

# Ativar o ambiente virtual
# No Linux/macOS:
source venv/bin/activate

# No Windows (PowerShell):
.\venv\Scripts\Activate.ps1

# No Windows (CMD):
venv\Scripts\activate.bat
```

7.2 Instalação das Dependências Python

Com o ambiente virtual ativado, instale as dependências:

```
pip install scikit-learn flask openpyxl joblib pandas
```

Ou, alternativamente, sem ambiente virtual (instalação global):

```
pip3 install scikit-learn flask openpyxl joblib pandas
```

7.3 Versões das Dependências Python

Biblioteca	Versão Mínima	Finalidade
scikit-learn	1.3+	TF-IDF Vectorizer + Naive Bayes
Flask	2.3+	Servidor HTTP para API de classificação
openpyxl	3.1+	Leitura do dataset .xlsx
jolib	1.3+	Serialização/desserialização do modelo
pandas	2.0+	Manipulação do dataset

8. Treinamento do Modelo de Machine Learning

O modelo precisa ser treinado antes da primeira execução. O arquivo `model.pkl` já está incluído no repositório (treinado com o dataset de 100 amostras), mas caso queira retreinar:

```
# No diretório ml/ (com ambiente virtual ativado, se aplicável)
cd ml
python3 train_model.py
```

O script executa as seguintes etapas:

1. Carrega o dataset `incidentes_cybersecurity_100.xlsx` (100 amostras, 5 categorias, 20 por categoria).
2. Concatena título e descrição em um único campo de texto.
3. Normaliza o texto (lowercase, strip).
4. Treina o pipeline TF-IDF + Naive Bayes com cross-validation de 5 folds.
5. Salva o modelo serializado em `ml/model.pkl`.
6. Salva as métricas de desempenho em `ml/metrics.json`.

A saída esperada no terminal é:

```
[ML] Carregando dataset...
[ML] Dataset carregado: 100 amostras
[ML] Distribuição de categorias:
phishing                20
malware                 20
brute_force             20
ddos                    20
vazamento_de_dados     20
[ML] Acurácia CV (5-fold): 0.97 ± 0.06
[ML] Acurácia no treino: 1.00
[ML] Modelo salvo em: ml/model.pkl
[ML] Métricas salvas em: ml/metrics.json
```

Nota: O arquivo `model.pkl` já está presente no repositório. O retreinamento é necessário apenas se o dataset for atualizado ou se o arquivo for removido.

9. Inicialização dos Servidores Flask (ML e PDF)

O sistema utiliza dois servidores Flask independentes: um para classificação ML (porta 5001) e outro para geração de relatórios PDF (porta 5002). Ambos devem estar em execução antes de iniciar o servidor Node.js.

9.1 Iniciar o Servidor de Classificação ML (Porta 5001)

Abra um **novo terminal** (mantenha este terminal aberto durante toda a sessão de uso):

```
# Navegar para o diretório ml
cd incident_security_system/ml

# Ativar ambiente virtual (se criado)
source venv/bin/activate # Linux/macOS
# ou: .\venv\Scripts\Activate.ps1 (Windows)

# Iniciar o servidor Flask de classificação
python3 classifier_server.py
```

A saída esperada é:

```
[Classifier] Carregando modelo de /caminho/para/ml/model.pkl...  
[Classifier] Modelo carregado com sucesso!  
* Running on http://127.0.0.1:5001  
* Debug mode: off
```

9.2 Iniciar o Servidor de Geração de PDF (Porta 5002)

Abra um **terceiro terminal** (mantenha este terminal aberto durante toda a sessão de uso):

```
# Navegar para o diretório ml  
cd incident_security_system/ml  
  
# Ativar ambiente virtual (se criado)  
source venv/bin/activate    # Linux/macOS  
  
# Instalar ReportLab (se ainda não instalado)  
pip install reportlab  
  
# Iniciar o servidor Flask de PDF  
python3 pdf_server.py
```

A saída esperada é:

```
[PDF Server] Iniciando servidor de geração de relatórios...  
* Running on http://127.0.0.1:5002  
* Debug mode: off
```

9.3 Verificar o Servidor de PDF

```
curl http://localhost:5002/health
```

Resposta esperada:

```
{
  "service": "pdf-generator",
  "status": "ok"
}
```

9.4 Verificar o Servidor de Classificação ML

Em outro terminal, verifique se o servidor está respondendo:

```
curl http://localhost:5001/health
```

Resposta esperada:

```
{
  "model_loaded": true,
  "metrics": {
    "cv_accuracy_mean": 0.97,
    "dataset_size": 100,
    "method": "TF-IDF + Naive Bayes (MultinomialNB)"
  },
  "status": "ok"
}
```

9.5 Testar a Classificação

```
curl -X POST http://localhost:5001/classify \
  -H "Content-Type: application/json" \
  -d '{"title": "E-mail suspeito", "description": "Recebemos e-mail com link malicioso solicitando credenciais de acesso ao sistema."}'
```

Resposta esperada:

```
{
  "category": "phishing",
  "confidence": 0.8921,
  "risk_level": "high",
  "probabilities": {
    "phishing": 0.8921,
    "malware": 0.0412,
    "brute_force": 0.0289,
    "ddos": 0.0198,
    "vazamento_de_dados": 0.0180
  }
}
```

10. Inicialização do Servidor Node.js

Com os dois servidores Flask em execução (portas 5001 e 5002), abra um **quarto terminal** e inicie o servidor de desenvolvimento Node.js:

```
# Na raiz do projeto
cd incident_security_system
pnpm dev
```

A saída esperada é:

```
> incident_security_system@1.0.0 dev
> NODE_ENV=development tsx watch server/_core/index.ts

[OAuth] Initialized with baseURL: https://oauth.seudominio.com
Server running on http://localhost:3000/
```

O servidor Vite (frontend) e o servidor Express (backend/API) são iniciados simultaneamente na porta 3000. O Vite serve o frontend com Hot Module Replacement (HMR) para desenvolvimento.

11. Verificação do Sistema

Com ambos os servidores em execução, abra o navegador e acesse:

```
http://localhost:3000
```

11.1 Checklist de Verificação

Realize os seguintes testes para confirmar que o sistema está funcionando corretamente:

Teste	Ação	Resultado Esperado
Página inicial	Acessar <code>http://localhost:3000</code>	Landing page SOC Portal carregada
Registro	Criar conta com e-mail e senha válida (ex.: <code>Senha@123</code>)	Redirecionamento para Dashboard
Validação de senha	Tentar registrar com senha <code>abc123</code> (sem maiúscula e sem especial)	Botão bloqueado, checklist exibido com critérios pendentes
Login	Fazer login com a conta criada	Dashboard exibido com 0 incidentes
Novo incidente	Registrar incidente com título e descrição	Incidente criado com categoria e risco
Classificação ML	Verificar categoria do incidente criado	Categoria atribuída automaticamente
Listagem	Acessar “Meus Incidentes”	Incidente criado listado
Filtros	Aplicar filtro por categoria	Lista filtrada corretamente
Análise de risco	Acessar “Análise de Risco”	Score e gráficos exibidos
Exportação PDF	Clicar em “Exportar PDF” na listagem	Download do PDF iniciado automaticamente
Painel Admin	Acessar <code>/admin</code> como admin	Listagem global de incidentes e usuários
Reclassificação	Reclassificar um incidente no admin	Categoria e risco atualizados
Logout	Clicar em logout	Redirecionamento para página inicial
Req. 6.5 Rate Limiting	Enviar 11 requisições de login em sequência rápida	11ª retorna HTTP 429 com mensagem de erro

Teste	Ação	Resultado Esperado
Req. 6.7 Helmet	<code>curl -I</code> <code>http://localhost:3000/api/trpc/auth.me</code>	Cabeçalho <code>X-Powered-By</code> ausente; <code>X-Content-Type-Options: nosniff</code> presente
Req. 6.4 IDOR	Tentar acessar incidente de outro usuário via URL	Retorna 404, nunca 403
Req. 6.8 Timing	Login com e-mail inexistente vs. existente	Tempo de resposta similar em ambos os casos

11.2 Promover Usuário para Administrador

Após registrar o primeiro usuário, promova-o para administrador via SQL:

```
UPDATE users SET role = 'admin' WHERE email = 'seu@email.com';
```

Após a promoção, faça logout e login novamente. O item **Admin** aparecerá no menu lateral.

11.2 Verificar Logs

Para monitorar os logs do servidor em tempo real, observe o terminal onde `pnpm dev` está em execução. Logs de requisições, erros e eventos de autenticação são exibidos com timestamp.

12. Scripts Disponíveis

Todos os scripts são executados na raiz do projeto com `pnpm <script>`:

Script	Comando	Descrição
dev	pnpm dev	Inicia o servidor de desenvolvimento (Node.js + Vite HMR)
build	pnpm build	Compila o frontend (Vite) e o backend (esbuild) para produção
start	pnpm start	Inicia o servidor em modo produção (requer build antes)
test	pnpm test	Executa todos os testes Vitest (162 testes em 5 arquivos)
db:push	pnpm db:push	Gera e aplica migrações do schema Drizzle no banco
check	pnpm check	Verifica erros de TypeScript sem compilar
format	pnpm format	Formata todos os arquivos com Prettier

13. Estrutura de Diretórios

```
incident_security_system/
├─ client/                                # Frontend React
│   ├─ public/                            # Arquivos estáticos (favicon, robots.txt)
│   │   └─ index.html                    # HTML raiz com fontes Google
│   └─ src/
│       ├─ App.tsx                       # Roteamento principal (Wouter)
│       ├─ index.css                     # Design SOC Portal (CSS variables OKLCH)
│       ├─ main.tsx                      # Ponto de entrada React
│       ├─ components/
│       │   └─ CyberLayout.tsx          # Layout SOC Portal com sidebar compacta
│       └─ pages/
│           ├─ Home.tsx                 # Landing page
│           ├─ Login.tsx                # Página de login
│           ├─ Register.tsx             # Página de registro
│           ├─ Dashboard.tsx            # Painel principal
│           ├─ Incidents.tsx            # Listagem com filtros
│           ├─ NewIncident.tsx          # Formulário de novo incidente
│           ├─ IncidentDetail.tsx       # Detalhe do incidente
│           ├─ RiskAnalysis.tsx         # Análise de risco
│           └─ Admin.tsx                # Painel de administração (role=admin)
│       └─ components/
│           ├─ CyberLayout.tsx          # Layout SOC Portal com sidebar compacta
│           └─ ExportPdfButton.tsx      # Botão reutilizável de exportação PDF
│       └─ lib/
│           └─ trpc.ts                  # Cliente trPC tipado
├─ drizzle/
│   └─ schema.ts                        # Schema do banco (users + incidents)
├─ ml/                                  # Motor de Machine Learning e PDF
│   ├─ train_model.py                   # Script de treinamento TF-IDF + Naive Bayes
│   └─ classifier_server.py             # Servidor Flask de classificação (porta
5001)
│   ├─ pdf_server.py                   # Servidor Flask de geração PDF (porta 5002)
│   ├─ model.pkl                       # Modelo treinado serializado
│   └─ metrics.json                    # Métricas de desempenho do modelo
│   └─ incidentes_cybersecurity_100.xlsx # Dataset de treinamento
├─ server/                             # Backend Node.js
│   └─ routers.ts                      # Procedures trPC (auth + incidents + admin
+ reports)
│   ├─ db.ts                           # Helpers de acesso ao banco (Drizzle)
│   ├─ validation.ts                   # Schemas Joi de validação
│   └─ incidents.test.ts               # Testes Vitest (79 testes: auth,
incidentes, admin, PDF, notif., design system)
```

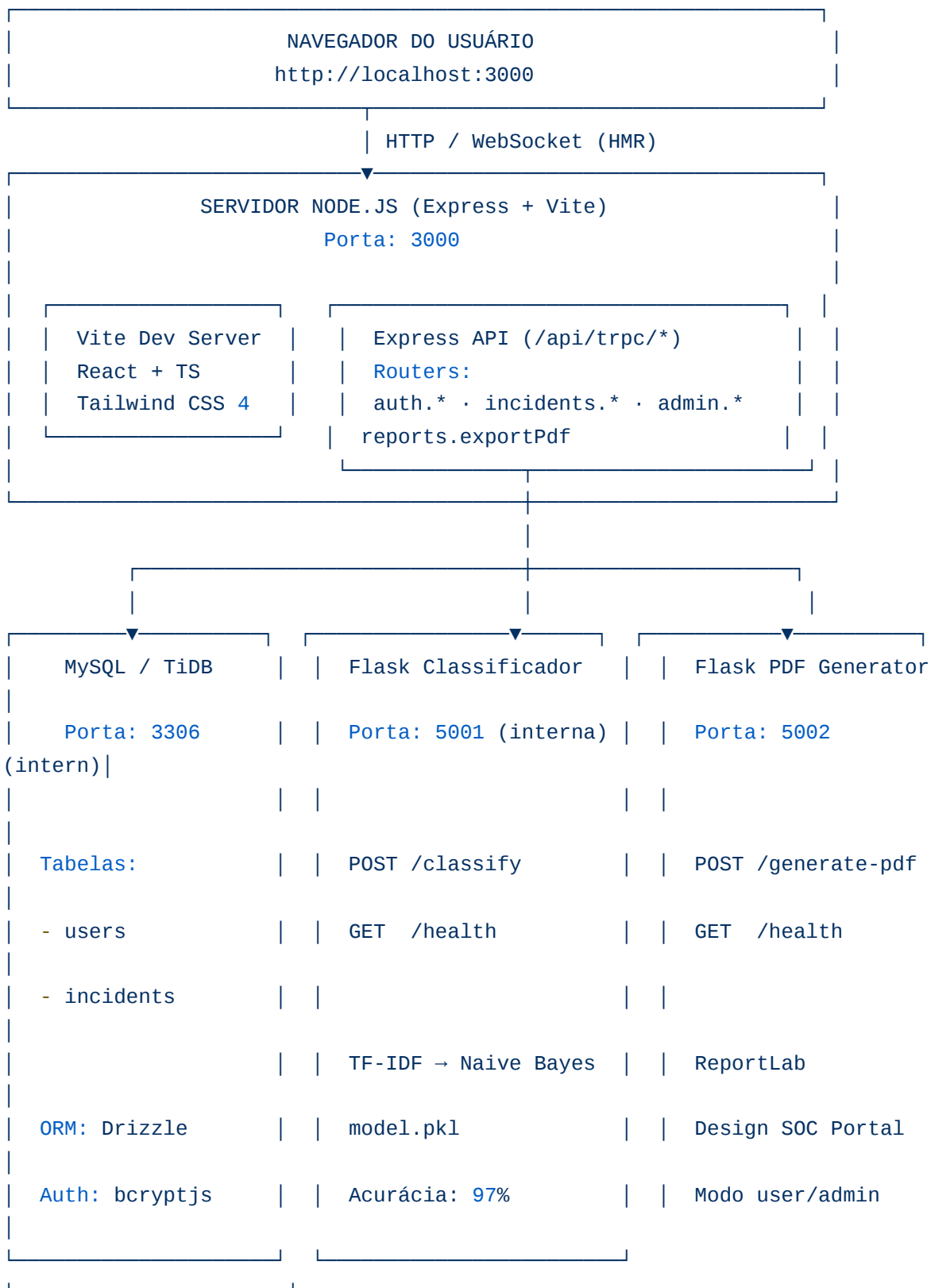
```

|   └─ auth.logout.test.ts      # Teste de logout com cookie sameSite:lax (1
teste)
|   └─ security.test.ts         # Testes dos 8 requisitos de segurança (34
testes)
|   └─ categories.test.ts       # Testes CRUD de categorias e RBAC (21
testes)
|   └─ recommendations.test.ts  # Testes recomendações de segurança
contextualizadas (27 testes)
|   └─ security.ts              # Middlewares: rate-limit, CORS, helmet
|   └─ _core/                   # Infraestrutura do framework
|       └─ index.ts             # Servidor Express principal
|       └─ context.ts           # Contexto tRPC (user, req, res)
|       └─ env.ts               # Variáveis de ambiente tipadas
|       └─ trpc.ts              # Configuração tRPC (public/protected)
|       └─ ...
└─ shared/
    └─ const.ts                 # Constantes compartilhadas (COOKIE_NAME)
└─ .env                         # Variáveis de ambiente (não versionado)
└─ drizzle.config.ts           # Configuração Drizzle Kit
└─ package.json                # Dependências e scripts
└─ pnpm-lock.yaml              # Lockfile de dependências
└─ todo.md                     # Rastreamento de funcionalidades
└─ tsconfig.json               # Configuração TypeScript
└─ vite.config.ts              # Configuração Vite

```

14. Arquitetura de Serviços

O sistema opera com **quatro processos independentes** que se comunicam internamente:



Importante: Os servidores Flask (portas 5001 e 5002) são exclusivamente internos. Eles não devem ser expostos publicamente. Toda comunicação passa pelo servidor

Node.js, que atua como proxy seguro.

15. Solução de Problemas Comuns

Problema: DATABASE_URL is required

Causa: O arquivo `.env` não foi criado ou a variável `DATABASE_URL` está ausente.

Solução: Verifique se o arquivo `.env` existe na raiz do projeto e se a variável `DATABASE_URL` está corretamente definida com a connection string do MySQL.

```
# Verificar se o arquivo .env existe
ls -la .env

# Verificar o conteúdo (sem expor senhas)
grep "DATABASE_URL" .env
```

Problema: Access denied for user (MySQL)

Causa: O usuário MySQL não tem permissão para acessar o banco de dados especificado.

Solução: Conecte-se ao MySQL como root e verifique os privilégios:

```
SHOW GRANTS FOR 'incident_user'@'localhost';
GRANT ALL PRIVILEGES ON incident_security.* TO 'incident_user'@'localhost';
FLUSH PRIVILEGES;
```

Problema: ECONNREFUSED 127.0.0.1:5001 (Servidor ML não está rodando)

Causa: O servidor Flask de classificação não foi iniciado antes do servidor Node.js.

Solução: Abra um terminal separado, navegue até o diretório `ml/` e execute `python3 classifier_server.py`. Mantenha este terminal aberto durante toda a sessão.

Problema: `ECONNREFUSED 127.0.0.1:5002` (Servidor PDF não está rodando)

Causa: O servidor Flask de geração de PDF não foi iniciado.

Solução: Abra um terminal separado, navegue até o diretório `ml/` e execute `python3 pdf_server.py`. Mantenha este terminal aberto durante toda a sessão.

Problema: `ModuleNotFoundError: No module named 'reportlab'`

Causa: A biblioteca ReportLab não foi instalada.

Solução:

```
pip3 install reportlab
```

Problema: `ModuleNotFoundError: No module named 'sklearn'`

Causa: As dependências Python não foram instaladas.

Solução:

```
pip3 install scikit-learn flask openpyxl joblib pandas
```

Se estiver usando ambiente virtual, certifique-se de que ele está ativado antes de instalar.

Problema: `model.pkl not found`

Causa: O arquivo do modelo não está presente no diretório `ml/`.

Solução: Execute o script de treinamento para gerar o modelo:

```
cd ml
python3 train_model.py
```

Problema: Porta 3000 já em uso

Causa: Outro processo está utilizando a porta 3000.

Solução:

```
# Linux/macOS: identificar o processo
lsof -i :3000

# Encerrar o processo (substitua PID pelo número encontrado)
kill -9 PID

# Windows: identificar o processo
netstat -ano | findstr :3000

# Encerrar o processo (substitua PID pelo número encontrado)
taskkill /PID PID /F
```

Problema: pnpm: command not found

Causa: O pnpm não está instalado.

Solução:

```
npm install -g pnpm
```

Problema: Erros de TypeScript no pnpm dev

Causa: Dependências desatualizadas ou cache corrompido.

Solução:

```
# Limpar cache e reinstalar
rm -rf node_modules
pnpm install
pnpm dev
```

Problema: Migrações falham com `Table already exists`

Causa: O banco de dados já possui tabelas de uma execução anterior.

Solução: O Drizzle utiliza migrações incrementais. Se as tabelas já existem com o schema correto, não é necessário reexecutar. Para forçar uma recriação (perda de dados):

```
DROP DATABASE incident_security;
CREATE DATABASE incident_security CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
```

Em seguida, execute novamente `pnpm db:push`.

16. Divisão de Atividades do Grupo

O sistema foi desenvolvido de forma colaborativa por uma equipe de cinco integrantes. A tabela abaixo apresenta a divisão de responsabilidades por área, relevante para fins de manutenção e suporte técnico:

Área	Integrante(s)	Artefatos sob Responsabilidade
Front-end	Nattan e Keven	client/src/pages/ , client/src/components/CyberLayout.tsx , client/src/components/ExportPdfButton.tsx , client/src/index.css , client/index.html
Back-end	Margefson	server/routers.ts , server/db.ts , server/validation.ts , server/security.ts (rate-limit, CORS, helmet), server/incidents.test.ts (79 testes), server/auth.logout.test.ts (1 teste), server/security.test.ts (34 testes — requisitos 6.1 a 6.8), server/categories.test.ts (21 testes — CRUD de categorias e RBAC)
Banco de Dados	Nattan	drizzle/schema.ts , drizzle.config.ts , migrações em drizzle/migrations/ , helpers de consulta em server/db.ts
Classificador ML	Josias e Keven	ml/train_model.py , ml/classifier_server.py , ml/pdf_server.py , ml/model/ , ml/metrics.json , ml/incidentes_cybersecurity_100.xlsx

Manual de Implantação gerado em Abril de 2026. Para suporte técnico, consulte o repositório: https://github.com/margefson/incident_security_system